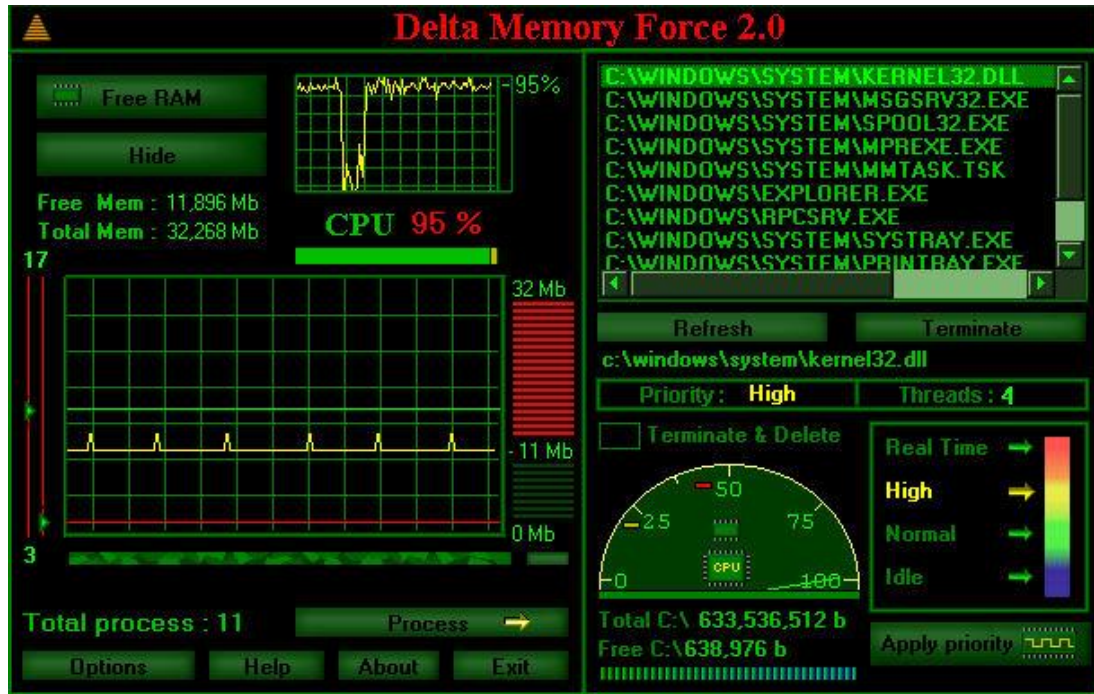


# Delta Memory Force

## *The Trial System and the Complete Key Generation, Verification, and Storage Mechanism*



*Delta Memory Force, the application's main interface*

### **Technical analysis based on the original Visual Basic 6 source code preserved in the project archives**

Main version analysed: DMF V1.0 (EN, 19 March 2002)

Key generator analysed: DMF TRIAL KEY

**Paul A. Gagniuc**

2026

## Purpose of This Document

**This document provides an explicit and complete description of the trial mechanism and licensing system implemented in Delta Memory Force.** The analysis follows the original source code and identifies the files involved, the Visual Basic 6 forms and controls, the values created in the Windows Registry, the auxiliary file stored in the Windows directory, the Product ID generation algorithm, the private key generator, the licence-verification procedure, and the conditions under which the application treated a copy as valid or expired.

**Terminological note.** The term “Key” is used in the code and interface for several different values. For clarity, this document distinguishes three concepts: the random internal installation code, the encrypted Product ID displayed to the customer, and the final activation key sent to the purchaser.

## 1. Archives and Files Analysed

| Archive or file                | Role in the system                                                                                                                                                          |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DMF V1.0 (EN - 19.03.2002).zip | Contains the source code of the application distributed to the customer, including the trial system, purchase form, and licence validation.                                 |
| DMF TRIAL KEY.zip              | Contains the private Product ID and key generator, the DMF-key.exe executable, and the access password.                                                                     |
| GraficOptim2.vbp               | The Visual Basic 6 application project. It declares Form2 as the start-up object and produces the g2.exe executable.                                                        |
| Security.bas                   | The central security module. It initialises the installation, generates the Product ID, verifies the licence, manages the trial counters, and checks the System32.pga file. |
| ScreenOptim2.frm               | The start-up and activation form. It contains the name and key fields, the Register button, and the call to the Transforma procedure. Its internal name is Form2.           |
| Formular.frm                   | The commercial form that collects the Product ID, customer details, and Western Union MTCN, then creates DeltaMemoryForce.txt. Its internal name is VVV.                    |
| key.vbp                        | The separate private-generator project. Its start-up object is Form2 from KeiPar.frm.                                                                                       |
| KeiPar.frm                     | The generator authentication form. It verifies the password PaulGagniucNOS.                                                                                                 |
| Key.frm                        | The generator interface and algorithm. It decodes the Product ID, constructs the activation key, and can verify a key before delivery. Its internal name is Form1.          |
| DMF-key.exe                    | The compiled executable of the private key generator.                                                                                                                       |
| readme.txt                     | Explicitly stores the generator password: PaulGagniucNOS.                                                                                                                   |

## 2. Application Start-up and Entry Point

The GraficOptim2.vbp project file declares Form2 as the start-up object:

```
Startup="Form2"
```

Form2 is the internal name of the ScreenOptim2.frm form. When loaded, it prevents a second instance of the application from starting and then calls the main security procedure:

```
If App.PrevInstance Then End
Transforma
```

The Transforma procedure is located in Security.bas. In sequence, it determines whether the executable appears to have been modified, whether a valid licence exists, whether the trial installation state is coherent, whether a new installation must be created, and whether the permitted number of launches has been exceeded.

### 3. Executable Size Verification

The first check in Security.bas measures the size of the current executable using Scripting.FileSystemObject:

```
fsiz = ShowFileSize(App.Path & "\" & App.EXENAME & ".exe")
If fsiz > 627100 Then End
```

If the executable exceeds 627100 bytes, the application terminates. The check does not use a hash and does not require an exact size match. It acts as a rudimentary threshold intended to detect certain modifications or additions made to the binary.

### 4. Windows Registry Storage Convention

The code uses the Visual Basic 6 SaveSetting and GetSetting functions. These functions store per-user values under the following structure:

```
HKEY_CURRENT_USER\Software\VB and VBA Program Settings\<AppName>\<Section>\<ValueName>
```

The exact AppName, Section, and ValueName strings constructed by the program are listed below, together with the purpose of each value.

#### 4.1. Explicit Licence Values

| Full path                                                                                 | Content and purpose                                                                                                              |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| HKEY_CURRENT_USER\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\Key         | Contains the encrypted Product ID of the installation. The name “Key” is misleading: this value is not the final activation key. |
| HKEY_CURRENT_USER\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\FullProgram | Contains the activation key entered by the customer after it has been validated. It is not a simple True or False flag.          |
| HKEY_CURRENT_USER\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\Name        | Contains the text “ Register to <customer name>”, subsequently displayed in the interface.                                       |

#### 4.2. Disguised Installation Markers

| AppName      | Section | ValueName | Value | Full path and purpose                                                                                  |
|--------------|---------|-----------|-------|--------------------------------------------------------------------------------------------------------|
| windows32IBM | IBM     | error     | 6     | HKCU\Software\VB and VBA Program Settings\windows32IBM\IBM\error. First installation-integrity marker. |
| Reg32GPS     | WinGPS  | Sat       | 6     | HKCU\Software\VB and VBA Program Settings\Reg32GPS\WinGPS\Sat. Second installation-integrity marker.   |

The names are constructed in the code through Chr(...) concatenations, reducing their visibility in a simple text search of the executable. The program checks both the presence and consistency of the two markers. If one exists while the other is missing, the state is treated as inconsistent.

### 4.3. The Three Copies of the Trial Counter

| AppName    | Section   | ValueName | Code variable | Full path                                                              |
|------------|-----------|-----------|---------------|------------------------------------------------------------------------|
| Charlize   | Theron    | Kaiser    | rr            | HKCU\Software\VB and VBA Program Settings\Charlize\Theron\Kaiser       |
| Intel32Sys | microsoft | windows   | danu          | HKCU\Software\VB and VBA Program Settings\Intel32Sys\microsoft\windows |
| MOVALO     | modem     | load      | siguranta     | HKCU\Software\VB and VBA Program Settings\MOVALO\modem\load            |

During initialisation, all three values are created with the value 0. On each permitted launch, they are incremented simultaneously. The duplication is intended to detect the deletion or modification of a single copy.

```
SaveSetting "Charlize", "Theron", "Kaiser", "0"
SaveSetting "Intel32Sys", "microsoft", "windows", "0"
SaveSetting "MOVALO", "modem", "load", "0"
```

## 5. The Auxiliary System32.pga File

Security.bas obtains the Windows directory through the GetWindowsDirectory API and constructs the following path:

```
<Windows directory>\System32.pga
```

On a system installed in C:\Windows, the resulting path is:

```
C:\Windows\System32.pga
```

The file is not placed in the C:\Windows\System32 subdirectory. It is stored directly in the Windows directory and merely carries the name System32.pga. Its written content is:

```
System run
```

The file constitutes a third type of marker, separate from the Registry. The code does not explicitly apply the Hidden attribute. Its name, however, makes it resemble a system file. If the file is missing while the two Registry markers exist, the application can recreate it. If the file exists but the markers are absent, the program treats the state as modified and may terminate.

## 6. Conditions for Initialising a New Installation

The program creates a new trial installation only when it finds neither a Product ID, nor the markers, counters, or control file. The condition in the code simultaneously checks that the following values are empty:

- fostdanu, corresponding to windows32\IBM\IBM\error
- OglindaOglinjoara, corresponding to Reg32\GPS\WinGPS\Sat
- mm, the content read from System32.pga
- rr, the first copy of the counter
- siguranta, the third copy of the counter
- QDelta, the DeltaMemoryForce\Trial\Key value

If all of these are empty, the application generates the internal installation code, creates the Product ID, writes the System32.pga file, creates the two markers with the value 6, and initialises the three counters to 0.

## 7. Generation of the Random Internal Code

During initialisation, Security.bas constructs a random 20-character string. The declared character set is:

```
stru = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

The generation loop is:

```
For q = 1 To 20
    Randomize
    j = Int(34 * Rnd(34))
    If j = 0 Then GoTo 7
    g = g + Mid(stru, j, 1)
Next q
```

The variable g represents the plaintext internal installation code. Its length is 20 characters. Because of the expression `Int(34 * Rnd(34))` and the rejection of the value 0, positions 1 through 33 of the declared string are used. In practice, the characters X, Y, and Z cannot be selected in this version.

The internal code g is not displayed to the customer and is not stored in plaintext in the Registry.

## 8. Construction of the Product ID

The 20-character internal code is combined with the constant string:

```
DeltaMemoryForcePaul
```

The constant string is also 20 characters long. The transformation is performed character by character:

```
ty = "DeltaMemoryForcePaul"
we = g
For hh = 1 To Len(ty)
    dmf = Mid(ty, hh, 1)
    dd = Asc(dmf)
    fg = dd + Asc(Mid(we, hh, 1)) - 5
    gforce = gforce & Chr(fg)
Next hh
```

The formula for position i is:

```
ProductID[i] = Chr(Asc("DeltaMemoryForcePaul"[i]) + Asc(InstallationCode[i]) - 5)
```

The resulting gforce value is stored in:

```
DeltaMemoryForce\Trial\Key
```

This value is the encrypted Product ID displayed in the customer form. The algorithm is proprietary and reversible, based on arithmetic operations applied to ASCII codes. It is not a standardised cryptographic algorithm such as AES or RSA.

### 8.1. Relationship to the Hardware

In the analysed source code of DMF V1.0 dated 19 March 2002, the Product ID is not derived from the processor serial number, disk serial number, motherboard, or any other hardware identifier. It is derived from the random string created during initialisation. The licence is therefore tied to the installation that produced the Product ID, rather than directly to the physical components of the computer.

## 9. Trial-State Consistency Verification

Before incrementing the counters, the program compares the disguised values. Under normal conditions, the following must be true:

```
rr = danu = siguranta
```

The code first checks whether rr and danu are equal. It then tests combinations of rr, danu, and siguranta. If one value differs while the other two match, the application terminates. This logic detects the typical case in which the user has modified or deleted a single copy of the counter.

The windows32IBM and Reg32GPS markers must also be identical. If one is missing or their values do not match, the program stops execution or reconstructs the state only in the cases explicitly handled by the code.

### 9.1. The 30-Launch Limit

The trial limit is defined implicitly through the calculation and messages in the code. On each launch, the application calculates:

```
ttyy = 30 - rr
Form2.sec.Caption = "      Start up " & ttyy & " of 30"
```

After the message is displayed, rr is incremented by 1. If the value does not exceed 30, the new number is stored in all three locations:

```
rr = rr + 1
SaveSetting "Charlize", "Theron", "Kaiser", rr
SaveSetting "Intel32Sys", "microsoft", "windows", rr
SaveSetting "MOVALO", "modem", "load", rr
```

On the first launch, the counter is 0 and the interface displays “Start up 30 of 30”. With each subsequent execution, the remaining number decreases. When rr becomes greater than 30, the application enters the “Trial Complete” state.

### 9.2. Controls Displayed after Expiration

When the trial is complete, Security.bas makes the registration controls in ScreenOptim2.frm visible:

```
R1
Name69
Reg69
Plata
Cancel
Label4
Label5
Formular
Image1
```

The control named Plata is presented in the interface as the Register button. R1 receives the activation key, while Name69 receives the purchaser’s name.

## 10. Resistance to a Simple Trial Reset

The trial state is distributed across seven distinct elements:

- the Product ID stored in DeltaMemoryForce\Trial\Key
- the windows32IBM\IBM\error marker
- the Reg32GPS\WinGPS\Sat marker
- the Charlize\Theron\Kaiser counter
- the Intel32Sys\microsoft\windows counter

- the MOVALO\modem\load counter
- the <Windows directory>\System32.pga file

Deleting the explicit DeltaMemoryForce\Trial section does not remove the disguised counters or the auxiliary file. Deleting a single counter copy creates a mismatch. Deleting the file without removing the markers may cause it to be recreated or may trigger detection of an inconsistent state. The mechanism is not impossible to remove for an analyst who studies the complete source code, but it is considerably more elaborate than a single counter stored under an obvious Registry key.

## 11. Purchase Form and File Sent to the Author

The button that opens the commercial form executes VVV.Show. The VVV form is defined in Formular.frm. When the form is loaded, the Product ID is read directly from the Registry:

```
ss = GetSetting("DeltaMemoryForce", "Trial", "Key")
VVV.ID.Text = ss
```

| Control      | Meaning                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------|
| Nume         | Customer's first and last name. The code requires a space in order to consider that two name components have been entered. |
| ID           | The installation Product ID, automatically read from the Registry.                                                         |
| CodBanca     | The Western Union Money Transfer Control Number (MTCN).                                                                    |
| Oras         | The country and city of the transfer.                                                                                      |
| AdresaClient | The customer's e-mail address. The code checks for the presence of the @ character.                                        |
| Cale         | The path of the generated text file. The default value is C:\Windows\Desktop\DeltaMemoryForce.txt.                         |

The form creates the DeltaMemoryForce.txt file and writes the customer's name, Product ID, MTCN banking code, country and city, and e-mail address into it. The file is not sent automatically. The interface instructs the customer to send it manually to DeltaMemoryForce@Yahoo.com after completing the Western Union payment.

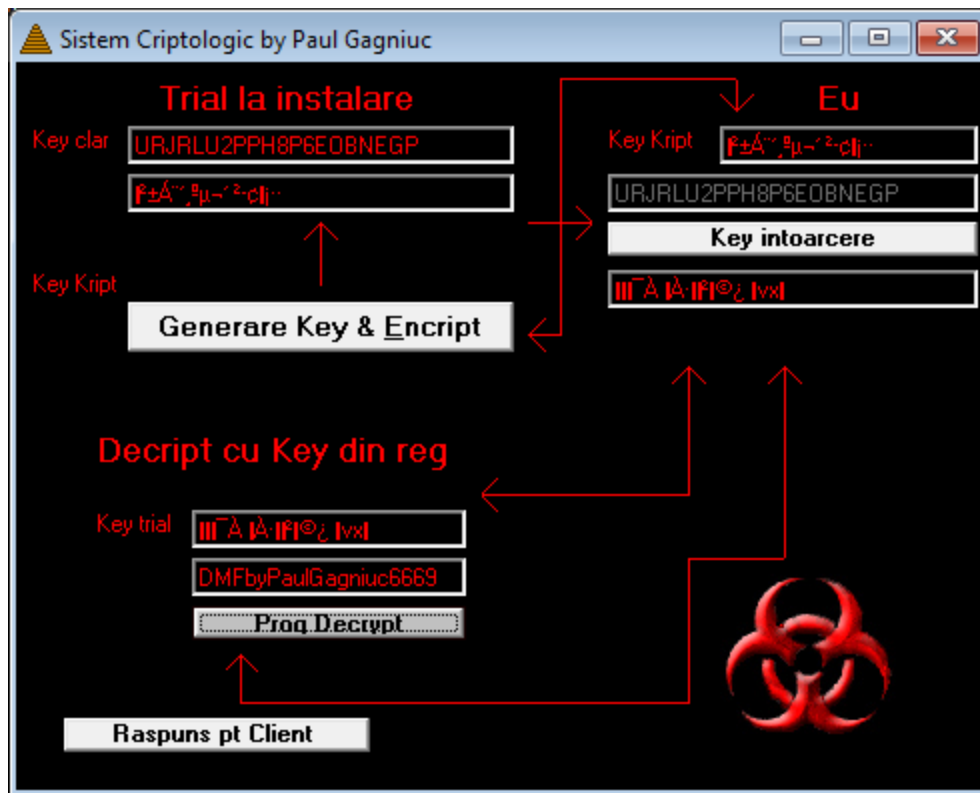
The text displayed in the form specifies a price of USD 25 and a period of 48 hours for delivery of the key by e-mail.

## 12. The Private Key Generator

The generator is contained in the key.vbp project and is compiled as DMF-key.exe. The project starts with Form2 from KeiPar.frm. The form displays a password field and checks the following string:

```
PaulGagniucN0S
```

The password is constructed in the code from Chr(...) characters and also appears explicitly in readme.txt. If the password is correct, the generator opens Form1 from Key.frm.



The private Product ID and key generator during a complete test

### 12.1. Main Controls in Key.frm

| Control        | Role                                                                          |
|----------------|-------------------------------------------------------------------------------|
| Text1          | The random internal code generated by the test function.                      |
| Text3          | The Product ID produced from the internal code.                               |
| Text5          | The Product ID received from the customer and entered to generate a real key. |
| Text4          | The internal code recovered by decoding the Product ID.                       |
| Text6          | The activation key generated for the Product ID in Text5.                     |
| Text9          | The key entered for verification.                                             |
| Text8          | The plaintext verification result, DMFbyPaulGagniuc6669, if the key is valid. |
| Command2_Click | Generates a random code and its corresponding Product ID for testing.         |
| Command4_Click | Decodes the Product ID and generates the final key.                           |
| Command5_Click | Verifies a key by applying the inverse operation.                             |

## 13. Decoding the Product ID in the Generator

The Product ID received from the customer is entered in Text5. Command4\_Click applies the inverse of the operation used in Security.bas:



```

tyt = Text5.Text
we = "DeltaMemoryForcePaul"
For hh = 1 To Len(tyt)
    dmf = Mid(tyt, hh, 1)
    dd = Asc(dmf)
    fg = dd - Asc(Mid(we, hh, 1)) + 5
    gforce69 = gforce69 & Chr(fg)
Next hh

```

The formula is:

```
InstallationCode[i] = Chr(Asc(ProductID[i]) - Asc("DeltaMemoryForcePaul"[i]) + 5)
```

The resulting gforce69 value is the original random internal code and is displayed in Text4.

## 14. Generation of the Final Activation Key

After recovering the internal code, the generator uses the fixed signature:

```
DMFbyPaulGagniuc6669
```

The signature is 20 characters long and is combined with the internal installation code:

```

ty = "DMFbyPaulGagniuc6669"
we = gforce69
For hh = 1 To Len(ty)
    dmf = Mid(ty, hh, 1)
    dd = Asc(dmf)
    fg = dd + Asc(Mid(we, hh, 1)) - 5
    gforce = gforce & Chr(fg)
Next hh
Text6.Text = gforce

```

The formula is:

```
ActivationKey[i] = Chr(Asc("DMFbyPaulGagniuc6669"[i]) + Asc(InstallationCode[i]) - 5)
```

The final key depends on the Product ID because the Product ID is decoded into the internal code, which directly participates in the calculation. The same Product ID produces the same key; different Product IDs produce different keys.

### 14.1. Customer Name and Key

The customer's name does not participate in the generation algorithm. The key depends on the Product ID, the recovered internal code, and the DMFbyPaulGagniuc6669 signature. The name is stored separately only after validation, so that the registration text can be displayed.

## 15. Key Verification in the Generator

Command5\_Click allows a key to be verified before it is sent. The generator reads the Product ID from Text5 and the test key from Text9. It first recovers the internal code from the Product ID and then decrypts the key using the inverse formula:

```
PlainCharacter = Chr(Asc(KeyCharacter) - Asc(InstallationCodeCharacter) + 5)
```

If the resulting string is exactly DMFbyPaulGagniuc6669, the key is valid and the result is displayed in Text8. Otherwise, the generator displays the message "Invalid Key - !".

## 16. Key Validation in the Customer Application

In ScreenOptim2.frm, the user enters the key in R1 and the name in Name69. The Plata button, displayed as Register, performs the following sequence:

1. Reads the entered key from R1 into the PaulGagniuc variable.
2. Reads the Product ID from DeltaMemoryForce\Trial\Key into the DeltaM variable.
3. Decodes the Product ID using the DeltaMemoryForcePaul string and recovers the internal installation code in gforce69.
4. Decodes the entered key using the recovered internal code.
5. Compares the result with the DMFbyPaulGagniuc6669 signature.
6. If the signature matches, it stores the key and name. If it does not match, it displays “Invalid Key!”.

The central fragment is:

```
DeltaM = GetSetting("DeltaMemoryForce", "Trial", "Key")
... Product ID decoding ...
... entered-key decoding ...
If gforce = "DMFbyPaulGagniuc6669" Then
    SaveSetting "DeltaMemoryForce", "Trial", "FullProgram", PaulGagniuc
    SaveSetting "DeltaMemoryForce", "Trial", "Name", rere
End If
```

## 17. Licence Storage and Reverification

After validation, the entered key is stored in full in the FullProgram value:

```
SaveSetting "DeltaMemoryForce", "Trial", "FullProgram", PaulGagniuc
```

The name is transformed into the text:

```
Register to <customer name>
```

and stored in the Name value. The application displays the message “Ok, restart the program!” and closes.

On subsequent launches, Security.bas does not treat the mere existence of the FullProgram value as sufficient proof. It repeats the cryptographic verification:

1. Reads FullProgram and Key.
2. Decodes the Product ID stored in Key using DeltaMemoryForcePaul.
3. Uses the recovered internal code to decode FullProgram.
4. Accepts the licence only if the result is DMFbyPaulGagniuc6669.
5. Reads Name and displays it in Form2.sec.Caption.
6. Executes Exit Sub so that the trial counters are no longer incremented.

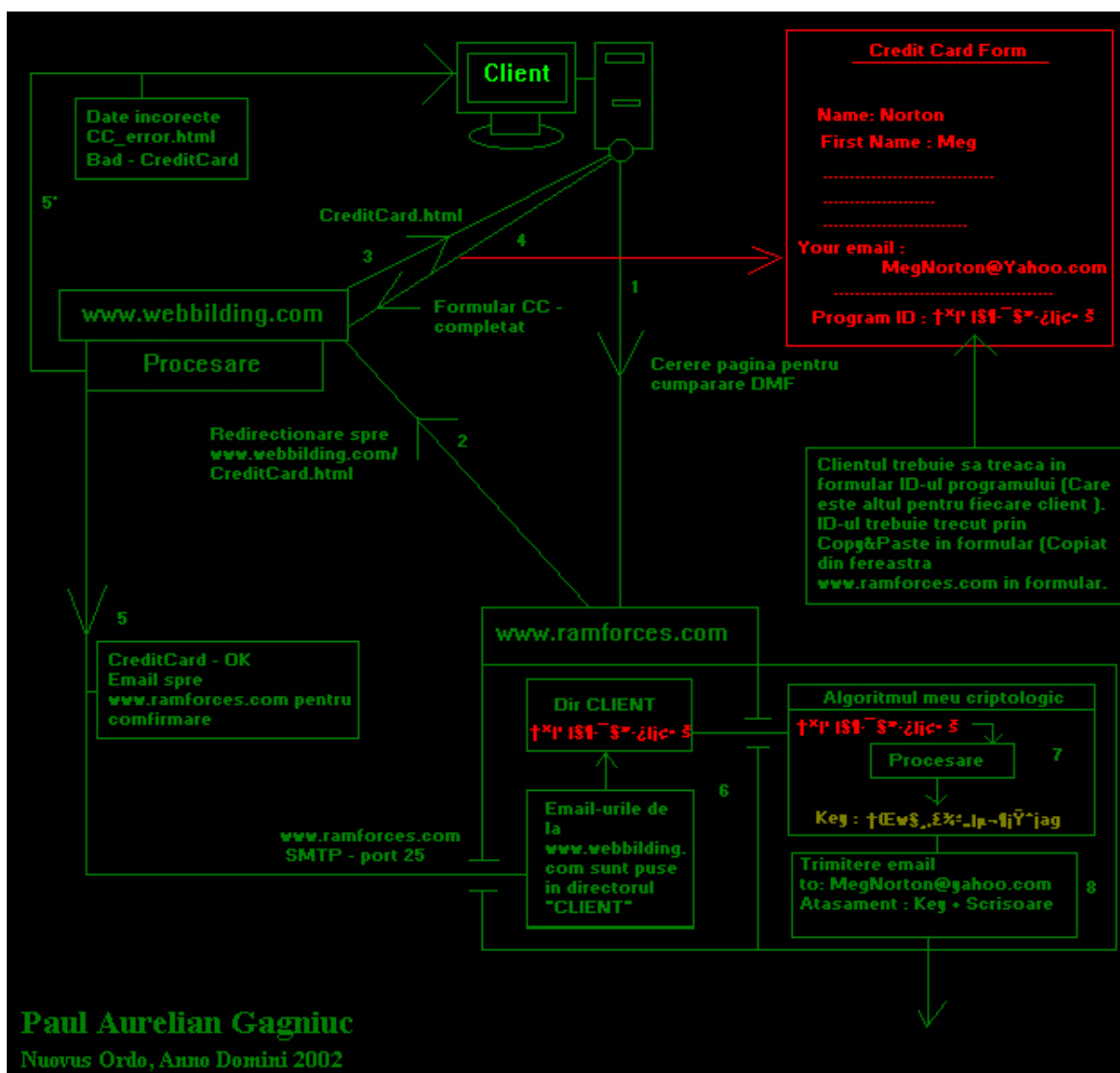
## 18. Complete Licensing Flow

```
1. First launch
   Random 20-character internal code
   ↓
2. Combination with DeltaMemoryForcePaul
   ↓
3. Product ID stored in DeltaMemoryForce\Trial\Key
   ↓
4. Product ID sent to the author
   ↓
5. The private generator recovers the internal code
   ↓
6. The internal code is combined with DMFbyPaulGagniuc6669
   ↓
7. The activation key is sent to the customer
```

- ↓
8. The customer enters the key and name
- ↓
9. The application decrypts the key and verifies the signature
- ↓
10. FullProgram and Name are stored in the Registry
- ↓
11. The licence is reverified on every launch

## 19. Commercial Context and Planned Automation

The generator archive also contains a diagram for a broader system covering purchase, confirmation, key generation, and e-mail delivery. The diagram documents the intention to connect online payment to the cryptographic generator and automatic key delivery. The source code analysed in this version, however, implements the manual flow through a form, a text file, and e-mail.



Original diagram of the commercial flow and automatic key generation

## 20. Exact Strings, Values, and Limits

| Element | Exact value |
|---------|-------------|
|---------|-------------|

|                                     |                                         |
|-------------------------------------|-----------------------------------------|
| Product ID string                   | DeltaMemoryForcePaul                    |
| Internal validation signature       | DMFbyPaulGagniuc6669                    |
| Generator password                  | PaulGagniucNOS                          |
| Internal-code length                | 20 characters                           |
| Trial limit                         | 30 launches                             |
| Executable-size threshold           | 627100 bytes                            |
| System32.pga content                | System run                              |
| Value of the two markers            | 6                                       |
| Initial value of the three counters | 0                                       |
| Price indicated in Formular.frm     | USD 25                                  |
| Specified key-delivery period       | 48 hours                                |
| E-mail address                      | DeltaMemoryForce@Yahoo.com              |
| File created for the customer       | C:\Windows\Desktop\DeltaMemoryForce.txt |

## 21. Technical Assessment in the Context of the Period

The system does not use standard algorithms such as RSA, AES, SHA, or digital signatures. The transformations are symmetric, reversible, and based on operations applied to ASCII codes. An analyst with access to the executable and sufficient time could reconstruct the algorithm.

For a shareware program created in the early 2000s, however, the mechanism is substantially more complex than a simple serial-number check. It combines:

- a different Product ID for each installation;
- a separate password-protected generator;
- a fixed internal signature, partially concealed through Chr expressions in the customer application;
- a key dependent on the Product ID;
- key reverification on every launch;
- three disguised copies of the trial counter;
- two additional Registry markers;
- an auxiliary file in the Windows directory;
- detection of inconsistencies between copies;
- a rudimentary executable-size check;
- a complete form for the Product ID, purchaser details, and payment confirmation.

The primary objective was not to create an unbreakable protection system, but to prevent a trivial trial reset, avoid a universal key, and preserve the author's control over each commercial activation.

## Appendix A. Central Algorithm Fragments

### A.1. Product ID Generation in Security.bas

```
stru = "0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ"
For q = 1 To 20
```

```

Randomize
j = Int(34 * Rnd(34))
If j = 0 Then GoTo 7
g = g + Mid(stru, j, 1)
Next q

ty = "DeltaMemoryForcePaul"
we = g
For hh = 1 To Len(ty)
    dmf = Mid(ty, hh, 1)
    dd = Asc(dmf)
    fg = dd + Asc(Mid(we, hh, 1)) - 5
    gforce = gforce & Chr(fg)
Next hh
SaveSetting "DeltaMemoryForce", "Trial", "Key", gforce

```

## A.2. Key Generation in Key.frm

```

tyt = Text5.Text
we = "DeltaMemoryForcePaul"
For hh = 1 To Len(tyt)
    dmf = Mid(tyt, hh, 1)
    dd = Asc(dmf)
    fg = dd - Asc(Mid(we, hh, 1)) + 5
    gforce69 = gforce69 & Chr(fg)
Next hh

ty = "DMFbyPaulGagniuc6669"
we = gforce69
For hh = 1 To Len(ty)
    dmf = Mid(ty, hh, 1)
    dd = Asc(dmf)
    fg = dd + Asc(Mid(we, hh, 1)) - 5
    gforce = gforce & Chr(fg)
Next hh
Text6.Text = gforce

```

## A.3. Key Validation in ScreenOptim2.frm

```

PaulGagniuc = R1.Text
DeltaM = GetSetting("DeltaMemoryForce", "Trial", "Key")

we = "DeltaMemoryForcePaul"
... recovery of the internal code in gforce69 ...

we = gforce69
ty = PaulGagniuc
... decoding of the key into gforce ...

If gforce = "DMFbyPaulGagniuc6669" Then
    SaveSetting "DeltaMemoryForce", "Trial", "FullProgram", PaulGagniuc
    SaveSetting "DeltaMemoryForce", "Trial", "Name", rere
End If

```

## Appendix B. Complete Persistent-Storage Map

| Type          | Path or file                                                                 | Value                         |
|---------------|------------------------------------------------------------------------------|-------------------------------|
| Product ID    | HKCU\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\Key         | Encrypted 20-character string |
| Activated key | HKCU\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\FullProgram | Entered and validated key     |

|                |                                                                        |                                                  |
|----------------|------------------------------------------------------------------------|--------------------------------------------------|
| Licence name   | HKCU\Software\VB and VBA Program Settings\DeltaMemoryForce\Trial\Name  | Register to <name>                               |
| Marker 1       | HKCU\Software\VB and VBA Program Settings\windows32\IBM\IBM\error      | 6                                                |
| Marker 2       | HKCU\Software\VB and VBA Program Settings\Reg32GPS\WinGPS\Sat          | 6                                                |
| Counter 1      | HKCU\Software\VB and VBA Program Settings\Charlize\Theron\Kaiser       | 0 at initialisation, then the number of launches |
| Counter 2      | HKCU\Software\VB and VBA Program Settings\Intel32Sys\microsoft\windows | 0 at initialisation, then the number of launches |
| Counter 3      | HKCU\Software\VB and VBA Program Settings\MOVALO\modem\load            | 0 at initialisation, then the number of launches |
| Auxiliary file | <Windows directory>\System32.pga                                       | System run                                       |

## Appendix C. Conclusion

**The Delta Memory Force mechanism was neither a simple trial counter nor a universal key.** It was a complete system distributed across the customer application, the Windows Registry, an auxiliary file, and a private generator. The Product ID concealed a unique random installation code. The generator recovered that code and used it to produce an installation-specific key. The application validated the key by recovering an exact internal signature, stored the key and purchaser's name, and repeated the validation on every launch. In parallel, the evaluation period was protected by three disguised counters, two markers, and the System32.pga file.